# Übungsblatt 9: Multicore & Finale (Klausurvorbereitung)

**Mikrocomputertechnik 3 (MCT3)**

## Zielsetzung

Sie vertiefen Vorlesung 9: Cache-Kohärenz / MESI, Race Conditions und RISC-V-Atomics (LR/SC) sowie eine kurze Klausur-Selbstkontrolle über Speicher, Interrupts, PWM und DMA.

**Bearbeitungszeit:** ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 8 P, Aufgabe 2: 8 P, Aufgabe 3: 8 P (Gesamt: 24 P)

## Teil 1: Theorie — Cache-Kohärenz und MESI

### Aufgabe 1.1: Veraltete Daten im SMP

In einem SMP-SoC hat jeder Kern einen privaten L1-Cache mit *Write-Back*.

1. Beschreiben Sie das Cache-Kohärenz-Problem, wenn Core 0 eine Variable ändert und Core 1 sie kurz darauf lesen will. Warum darf Core 1 nicht einfach den gemeinsamen RAM lesen?
2. Warum löst man das Problem per Hardware (Bus Snooping) statt durch manuelles Software-Flushing vor jedem Lesevorgang?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 1.2: MESI-Protokoll

1. Nennen Sie die vier MESI-Zustände und ihre Bedeutung.
2. Core 0 und Core 1 haben dieselbe Line beide in **S (Shared)**. Was passiert auf dem Bus, wenn Core 0 schreiben will? Welche Zustände haben die Lines danach in Core 0 und Core 1?

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

## Teil 2: Theorie — Race Conditions und Hardware-Atomics

### Aufgabe 2.1: Anatomie eines Software-Fehlers

Betrachten Sie `zaehler++;` auf einer globalen Variable.

1. Übersetzen Sie den Befehl logisch in die drei RMW-Schritte (Assembler-Ebene).
2. Erklären Sie chronologisch eine Race Condition: beide Kerne inkrementieren „gleichzeitig“, das Ergebnis steigt aber nur um 1 statt um 2 (Startwert 0).

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 2.2: RISC-V Atomics (LR/SC)

Software-Locks wie `while(lock==1); lock=1;` scheitern, weil Prüfen und Setzen selbst racy sind. RISC-V nutzt `lr.w` / `sc.w`.

1. Erklären Sie `lr.w` und `sc.w`. Welche Rolle hat der Hardware-Monitor?
2. Was passiert, wenn Core 1 `sc.w` ausführt, nachdem Core 0 die Adresse bereits verändert hat?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

## Teil 3: Systemverständnis (Klausur-Wrap-Up)

*Kurze Selbstkontrolle quer über das Semester.*

### Aufgabe 3.1: Speicher und Caches

1. Warum führt Column-Major-Zugriff auf ein 2D-Array in C zu starkem Thrashing, Row-Major aber nicht?
2. Unterschied Direct Mapped vs. 4-Way Set-Associative bei der Platzwahl eines neuen Blocks?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 3.2: Interrupts und Timer

1. Architektonischer Unterschied CLINT vs. PLIC? Je ein typisches Einsatzgebiet.
2. Warum darf eine ISR nicht wie eine normale C-Funktion mit `ret` enden (`__attribute__((interrupt))`, `mret`)?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 3.3: DMA und Aktorik

1. Rollen von `ARR` und `CCR` bei PWM?
2. Was ist ein Interrupt Storm beim SPI-Senden großer Datenmengen per IRQ — und warum hilft DMA?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 3.4: AMAT-Wiederholung

Hit Time = 1 Takt, Miss Rate = 5 %, Miss Penalty = 100 Takte.

1. Geben Sie die AMAT-Formel an und berechnen Sie die AMAT.

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*